

《软件工程》开卷历年题目整理及答案

21 软件工程 厉秋实 陈宇森

第一套 P7-12

1. 请从软件工程的角度，结合你的个人实践，说明计算机软件与程序二者之间有何区别（4 分）以及联系（2 分）？
2. “软件工程”强调要采用“工程的”的手段来进行软件系统的开发和维护，请结合你阅读开源软件的实践，阐述你是如何理解上述思想的？即软件工程中的“工程”手段是指哪些手段。
3. 请列举出反映程序质量低劣的 4 个方面基本表现（2 分）；并解释说明在编写程序过程中（2 分）以及编写完成之后（2 分）分别应该采用什么样的手段来克服程序质量低劣问题。
4. 通常程序员编写好程序之后需要根据所编写程序的具体设计要求来对自己编写的程序进行单元测试，包括设计测试用例、运行测试程序和发现程序中的故障等等。如果在测试过程中发现问题，程序员需要调试并修改程序以纠正错误。请考虑以下场景：程序员通过测试发现了某个程序中的一个故障并通过修改程序纠正了该故障，程序员认为已经完成了该程序的开发工作。请问程序员的这种认识和做法是否合理？如果合理请解释为什么？如果不合理请说明该如何去做？

以下有一段关于某个软件“用户注册”部分功能的需求描述，请分析该需求描述中存在的至少 4 处问题（4 分），并指明导致这些问题的原因（4）。基于对这些问题的分析和解决，给出改进后的需求描述（2 分）（注明：可以在现有需求描述的基础上自己想定需求）。

“用户在使用系统之前必须注册，用户注册必须提供用户名、账号和用户类别，并同时提供用户的手机号、邮箱地址、邮寄地址等方面的信息，用户名必须是全局唯一的，只能由字符和数字构成，用户名也可以用邮箱或者手机号表示；密码的字符串长度必须大于 6，注册成功后系统必须给用户返回信息，系统的注册功能必须快速完成，不用让用户等待太长时间”。

敏捷软件开发方法认为过度的依赖文档会使得软件开发非常“笨重”，难以快速应对变化，它建议适当编写文档，更加重视程序代码的编写，你认为敏捷方法的这一思想是否合理，请给出解释和说明。

根据软件工程的观点，软件是程序+数据+文档，而不仅仅是程序代码；请结合课程软件开发项目的实践，解释说明为什么软件项目的开发需要文档？（3 分）；请列举出 4 个常见的文档例子（2 分）

5. 请阅读以下程序代码，指出其代码编写不规范之处，并说明原因。回答格式可为：第 ** 行：存在什么问题。

第二套 P13-18

1. 请结合你阅读小米便签开源软件，介绍你在阅读过程中产生了哪些文档（2 分）？为什么需要这些文档（2 分）？这些文档在后续的软件开发中有何作用（2 分）
2. 测试(Testing)和调试(Debugging)均是软件开发过程中的二项重要工作，请说明这二项工作有何区别（2 分）？二者之间有何关系（2 分）？
3. 在阅读小米便签开源软件过程中，我们强调程序质量和编程风格对于开发高质量的软件的重要性。请结合你的课程实践，解释说明 1) 遵循编程风格规范可以从哪些方面提高软件系统的质量？（3 分） 2) 请列举 4 项常用的编程风格（3 分）
4. 请结合针对小米便签软件代码的阅读和维护，说明你们小组是如何进行/开展结对编程的（3 分），这种编程方式有何好处（3 分）
5. 请你分析为什么敏捷软件开发方法可以有效的应对软件需求的变化（3 分）？该方法采用哪些举措来应对软件需求的变化（3 分）？
6. 软件需求事先难以确定且在开发过程中会经常性的发生变化，这是软件项目的特点。请针对这一特点，分析一下问题： 1) 软件需求变化会对软件开发产生什么样的影响；（3 分） 2) 如果在开发过程中（如在软件设计阶段）软件需求发生了变化，该如何进行处理？（3 分）

第三套 P19-21

1. 制定软件项目计划要考虑以下三方面的要素：（1）软件开发过程；（2）要完成的工作；（3）约束和限制。请逐一解释为什么要考虑这些因素。
2. 请解释（1）迭代软件开发过程模型与增量软件开发过程模型二者之间有何区别；（2）软件开发过程模型与软件项目计划二者之间有何区别？
3. “易变性”是软件开发的特点，它是指用户针对软件的需求经常会发生变化，甚至在开发的后期，请结合你个人的软件开发实践以及对开源软件的阅读，谈谈以下问题：
 - 1) 软件系统的需求为什么具有“易变性”的特点；（2 分）
 - 2) 分别从技术和管理二个角度分析，软件需求的变化会对软件项目开发产生什么样的影响；（3 分）
 - 3) 结合对开源软件的阅读，请谈谈 OSChina 程序代码有哪些好的举措来应对软件需求的易变性。（3 分）
4. 请结合本课程的讲授，谈谈“程序设计”和“软件开发”二者之间的异同

第四套 P22-25

1. 请结合小米便签阅读和维护课程实践，解释说明（1）编写代码时为什么要遵循编码规范？（2）谁会关注代码编写的规范性？（3）结合例子说明，小米便签程序代码体现了哪些高质量的软件设计原则。
2. 在软件开发过程中如果软件需求发生了变化，会对该软件系统的哪些软件制品产生影响？结合小米便签开源软件的维护，解释说明你们如何根据变化的软件需求来开展软件开发工作的？（对实现增加的功能进行需求分析，在已有功能的基础上思考如何让用户更加满意）
3. 软件迭代过程模型和敏捷开发方法是否都能有效应对软件需求的变化，如果不行说明原因，如果可以解释二者在应对变化方面的差别？
4. 说明面向对象软件开发方法提供了哪些技术手段来支持软件重用？
5. 以下是某个模块的功能描述及接口设计，请采用等价分类法为该模块的集成测试设计测试用例，详细说明测试用例设计的步骤以及最终的测试用例结果。

Integer: QueryRemainingSeat(TrainNo, CoachClass), 该模块根据输入的高铁车次（假设只有三个车次，分别是：G80、G21、G1034）和座位类别（商务座、一等座、二等座），查询当日该车次及该座位等级剩余的车票数量。如果输入信息不合法，则返回结果为-1。

第五套 P26-29

1. 请结合你的课程实践，列举四类软件开发文档（2 分），解释软件开发为什么需要文档（2 分）？分析软件文档会给软件开发带来哪些负面问题（2 分）
2. 请结合课程实践，解释和说明 Git 工具在软件开发中起到什么作用（2 分）？它主要提供了哪些功能（2 分）？
3. 请提出 3 种可以提高软件系统质量的软件工程方法（3 分），并解释他们为什么可以提高软件开发的质量（3 分）？
4. 软件开发初期要完整、准确地获取软件需求是非常困难的，甚至是不可能的。在软件开发过程中，软件需求会发生变化。请说明软件需求的变化会对软件开发产生什么样的影响？（3 分）？在这种情况下会对软件设计提出什么样的要求？（3 分）
5. 对现有软件开发项目的情况进行统计，可以发现软件维护的形式多样，成本很高，周期很长。请结合这一现象，阐述在软件开发阶段应该采取哪些举措来应对软件维护的上述挑战和问题？（6 分）
6. 以下是一段 12306 火车票购买软件的需求描述，刻画了旅客查询和订购火车票的需求信息，请针对该需求描述（1）绘制出 user case 模型（4 分）；（2）分别绘制出查询火车票和订购火车票的顺序图（8 分）；（3）绘制出针对该软件需求的类图（4 分）。

说明：要求正确使用 UML 的图形化模型以及各个图形符号

“用户输入“出发日期”、“出发时间段”、“出发站”、“到达站”、“火车类别”信息，系统将按照时间先后次序，列举出所有满足条件的火车车次信息，包括车次、出发站、始发站、到达站、目的站、出发时间、到达时间，剩余车票数量等等用户在购买车票之前需要登录系统，随后提供出发的日期、出发站、到达站信息，系统将根据上述信息查询出满足要求的车次，用户从中选择欲购买的车次，确定欲购买车票的数量，系统将告知需要用户支付的价格，并要求用户提供支付火车票的信用卡及其所属的银行；随着连接该银行的支付系统完成支付，支付成功后系统将确认车票购买成功，随后给用户发去成功购票的短信

第 6 套 P30-34

1. 说明软件测试与程序调试二者之间的区别（2 分）及联系（2 分）？
2. “软件工程”包含哪些基本要素（2 分）？这些要素如何支撑软件系统的工程化开发（2 分）？
3. 结合小米便签开源代码的阅读和维护，列举出小米便签代码质量有哪些好的方面（4 分）；存在哪些方面的不足（2 分）
4. 根据软件工程的思想，软件设计通常需要遵循以下的原则：模块化、分解、信息隐藏，从而提高软件系统的质量。请解释为什么这三项原则有助于提升软件设计质量（每个 2 分）
5. 请解释说明软件维护和软件开发二者之间的区别和联系（4 分）
6. 软件测试包括哪些工作（2 分）？为什么测试用例的设计对于软件测试而言非常重要（2 分）？请解释软件测试的原理（2 分）
7. 为什么要将软件测试分为单元测试、集成测试和确认测试等不同的阶段（2 分）？这些阶段通常分别由什么样的人来完成（2 分）？针对这些阶段的测试用例分别在软件开发的哪些阶段来生成（2 分）？为什么？（2 分）

第 7 套 P35-37

1. 结合阅读和标注小米便签开源软件实践，说明小米便签代码中有哪些“好”的软件开发实践和经验值得你学习？（3 分），小米便签代码还有哪些“不好”的质量表现？（3 分）
2. 为什么软件系统的需求容易发生变化？（3 分），请分析软件需求的变化会对软件开发活动和软件制品各产生什么样的影响（3 分）？
3. 请结合课程实践，说明在面向对象需求分析阶段，需求模型中“用例图”、“顺序图”、“分析类图”三者间保持一致性是指什么含义？（3 分）
4. 在软件需求分析阶段，质量保证对于成功地开发软件系统至关重要。请列举在需求分析阶段能够有效确保需求分析及其制品质量的举措有哪些？（3 分），并解释原因（3 分）。

第 8 套 P38-40

1. 请从软件工程师的角度，解释说明软件需求分析、软件设计、编码实现这三者的开发任务、输出制品有哪些差异性？
2. UML 提供了哪些模型可用来对软件系统的行为特征进行描述和建模？
3. 质量对于软件系统而言是非常重要的。请说明在软件开发过程中需要采用哪些方面的工程化手段来保证所开发软件系统的质量。
4. 针对 12306 软件系统，请采用 UML 进行建模和分析。
 - 1) 系统为用户提供用户注册、查询车次、购买车票、退票、改签车次等常用功能
 - 2) 用户购买车票的大体流程是：首先登录到系统之中，查询和选择车次，提供支付银行账号信息，随后系统将向用户发送短信验证码，用户输入验证码确认正确后，与银行系统进行交互，完成支付，最后完成购票业务，并给用户发送短信以反馈车票信息。

(10 分) 使用 UML 用例图对该系统的需求进行建模。

(10 分) 用 UML 顺序图对用户注册的业务流程进行建模。

(5 分) 基于用例图和顺序图，给出上述系统的 UML 类图。

第 9.10 套 P41-43

1. 请结合课程实践，解释说明软件文档和模型在软件开发过程中发挥了什么作用？
2. 软件开发过程中通常会产生三类软件制品：模型、文档和代码。请说明（1）这三类软件制品之间存在什么样的关系（2 分）；（2）在开发过程中要保持这三类制品之间的一致性是指什么含义（2 分）；（3）如何来保持它们之间的一致性（2 分）。
3. 软件测试能否发现程序中的所有错误？为什么？
4. 结合你的课程实践，解释说明 UML 的顺序图在分析解决和设计阶段分别用于对什么样的内容进行建模？所建立的模型有何用途？
5. 结合“小米便签”开源代码的阅读和分析，说明该开源代码有哪些“好”的软件设计经验（3 分）和“好”的程序设计经验（3 分）。

《软件工程》考试试卷卷 1

1. 简答题（6 分）

请从软件工程的视角，结合你的个人实践，说明计算机软件与程序二者之间有何区别（4 分）以及联系（2 分）？

区别：

- 概念层面：软件是一个更广泛的概念，它包括了程序以及与之相关的文档、数据等；而程序只是软件的一个组成部分，是软件的具体实现。
- 范围：软件可以包括多个程序，以及这些程序所依赖的库、配置文件等程序则是单一的可执行文件，只包含一组指令。
- 功能：软件通常具有完整的功能，可以解决特定问题；程序则是为了实现某个具体功能而编写的代码。

联系：

- 组成关系：程序是构成软件的基本单位，一个软件可能包含多个程序。
- 开发过程：软件开发过程中，程序员需要编写程序来实现软件的功能。在软件开发的阶段，程序可能会经历设计、编码、测试、维护等过程。
- 运行方式：软件的运行依赖于其中的程序，程序在计算机上执行时，会调用操作系统提供的服务，从而实现软件的功能。

2. 简答题（4 分）

“软件工程”强调要采用“工程的”的手段来进行软件系统的开发和维护，请结合你阅读开源软件的实践，阐述你是如何理解上述思想的？即软件工程中的“工程”手段是指哪些手段。

P58 关于软件工程的方法：系统的，规范的，可量化的

- 首先，软件工程中的“工程”手段强调了一种系统化、规范化的开发方法。这包括从需求分析、设计、编码、测试到维护的整个软件开发生命周期。在开源软件的实践中，我注意到优秀的项目往往有清晰的需求分析文档，明确的设计规范，以及详尽的测试用例。这些都有助于确保软件的质量和稳定性。
- 其次，“工程”手段也强调了团队协作的重要性。在开源世界中，许多项目是由全球范围内的开发者共同参与的。为了确保项目的顺利进行，开发者们需要遵循一定的协作规范，如代码风格、提交规范等。同时，有效的沟通手段，如邮件列表、即时通讯工具等，也是必不可少的。
- 此外，“工程”手段还体现在对软件复杂性的管理上。随着软件规模的扩大，复杂性也随之增加。为了应对这种挑战，软件工程师们会采用各种技术手段，如模块化、分层设计、微服务等，来降低软件的复杂性。在开源软件的实践中，我观察到许多项目通过这些手段有效地提高了代码的可维护性和可扩展性。
- 最后，“工程”手段还意味着持续的改进和优化。软件工程的目的是要开发出高质量的软件，而这一目标并不是一蹴而就的。为了达到这个目标，软件工程师们需要不断地进行代码审查、性能优化等工作。在开源软件的实践中，我注意到许多项目都有持续集成/持续部署（CI/CD）的流程，以确保软件始终处于最佳状态。

- 综上所述，软件工程中的“工程”手段主要包括系统化、规范化的开发方法，团队协作，复杂性管理以及持续的改进和优化等。在阅读开源软件的实践中，我深刻认识到这些“工程”手段对于提高软件质量、稳定性和可维护性的重要性。

2. 简答题（6 分）

请列举出反映程序质量低劣的 4 个方面基本表现（2 分）；并解释说明在编写程序过程中（2 分）以及编写完成之后（2 分）分别应该采用什么样的手段来克服程序质量低劣问题。

程序质量低劣的四个方面基本表现：

- 功能性问题：程序未能正确实现预期的功能，存在 bug 或者漏洞，无法满足用户需求。
- 可维护性差：代码结构混乱，缺乏合理的模块化和封装，导致后续维护困难。
- 性能问题：程序运行效率低下，响应时间长，资源消耗高，无法处理大量数据或并发请求。
- 可用性不足：用户界面不友好，操作复杂，用户体验差，难以上手使用。

在编写程序过程中应该采用的手段来克服程序质量低劣问题：

- 代码规范与重构：遵循编码规范，保持代码风格一致，定期进行代码重构以提高代码质量。
- 单元测试：编写单元测试来验证每个模块的功能，确保代码的正确性和稳定性。
- 设计模式：合理运用设计模式来解决特定问题，提高代码的可读性和可维护性。
- 性能分析与优化：在开发过程中进行性能分析，找出瓶颈并进行优化，以提高程序的运行效率。

编写完成之后应该采用的手段来克服程序质量低劣问题：

- 集成测试：进行集成测试以确保各个模块之间能够正确协作，没有接口问题。
- 系统测试：进行全面的系统测试，包括功能测试、性能测试、压力测试等，确保软件系统的稳定性和可靠性。
- 代码审查：组织代码审查会议，让团队成员相互检查代码，发现潜在的问题和改进点。
- 用户反馈：收集用户反馈，了解用户在使用过程中遇到的问题，及时进行修复和优化。

4. 简答题 (6 分)

通常程序员编写好程序之后需要根据所编写程序的具体设计要求来对自己编写的程序进行单元测试，包括设计测试用例、运行测试程序和发现程序中的故障等等。如果在测试过程中发现问题，程序员需要调试并修改程序以纠正错误。请考虑以下场景：程序员通过测试发现了某个程序中的一个故障并通过修改程序纠正了该故障，程序员认为已经完成了该程序的开发工作。请问程序员的这种认识和做法是否合理？如果合理请解释为什么？如果不合理请说明该如何去做？

软件测试技术在 p421

不合理：

- 回归测试：修改后的代码可能会引入新的错误，或者影响其他部分的功能。因此，仅仅修复了一个已知的错误并不意味着所有的问题都已经解决。程序员需要进行回归测试，以确保修改没有破坏原有的功能。
- 完整性检查：程序员需要确保所有的功能需求都被正确实现，并且所有的代码都已经过测试。有时候，一些问题可能只有在特定的条件下才会显现，因此在完成所有功能的测试之后，才能认为开发工作完成。
- 性能评估：即使功能上没有问题，程序的性能也可能不符合要求。程序员需要对程序进行性能测试，确保它在各种预期的工作负载下都能保持良好的性能。
- 代码审查：一个人可能难以发现自己代码中的所有问题。进行代码审查可以让其他开发者帮助检查代码，提高代码质量，发现潜在的错误和改进点。
- 用户验收测试：如果程序是为特定的客户或用户群体开发的，那么在最终交付之前，应该进行用户验收测试（UAT），以确保程序满足用户的实际需求。
- 文档更新：在程序修改后，相关的技术文档、用户手册等也需要更新，以反映最新的程序状态和使用方法。
- 持续集成：如果是在团队环境中工作，程序员应该将修改后的代码合并到主分支，并确保它能够通过持续集成系统中的所有自动化测试。

5. 分析题（10 分）

以下有一段关于某个软件“用户注册”部分功能的需求描述，请分析该需求描述中存在的至少 4 处问题（4 分），并指明导致这些问题的原因（4）。基于对这些问题的分析和解决，给出改进后的需求描述（2 分）（注明：可以在现有需求描述的基础上自己想定需求）。

“用户在使用系统之前必须注册，用户注册必须提供用户名、账号和用户类别，并同时提供用户的手机号、邮箱地址、邮寄地址等方面的信息，用户名必须是全局唯一的，只能由字符和数字构成，用户名也可以用邮箱或者手机号表示；密码的字符串长度必须大于 6，注册成功后系统必须给用户返回信息，系统的注册功能必须快速完成，不用让用户等待太长时间”。

关于软件需求的质量要求在 p142

问题：

- 需求描述中并未明确指出账号的作用和要求，可能会导致开发人员对账号的理解产生混淆。
- 用户类别的具体含义和分类没有明确说明，可能会导致开发人员在实现时无法确定如何设计和处理用户类别。
- 对于用户名的规定存在矛盾，一方面要求用户名只能由字符和数字构成，另一方面又允许用户名用邮箱或者手机号表示，这可能会导致开发人员在实现时产生困惑。
- 对于系统注册功能的速度要求没有具体的量化标准，“不用让用户等待太长时间”这样的表述过于模糊，无法为开发人员提供明确的指导。

原因：

- 需求描述中对账号的忽视可能是因为需求提出者认为账号的需求是显而易见的，但在实际情况中，不同的系统对账号的要求可能会有所不同，因此需要在需求描述中明确指出。

- 用户类别的具体含义和分类可能因为涉及到业务逻辑，需求提出者认为这是开发人员应该了解的内容，但在实际操作中，开发人员可能并不了解这些业务逻辑，因此需要在需求描述中明确说明。
- 对于用户名的规定存在矛盾可能是因为需求提出者在编写需求时没有充分考虑到所有的可能情况，导致规定之间产生了冲突。
- 对于系统注册功能的速度要求没有具体的量化标准可能是因为需求提出者认为这是一个通用的要求，但实际上，不同的系统对速度的要求可能会有所不同，因此需要在需求描述中给出具体的量化标准。

改进后的需求描述：“用户在使用系统之前必须注册，用户注册必须提供用户名、账号和用户类别，并同时提供用户的手机号、邮箱地址、邮寄地址等方面的信息。用户名必须是全局唯一的，只能由字符和数字构成，长度在 6-16 位之间，用户名也可以用邮箱或者手机号表示，但不能用特殊字符；密码的字符串长度必须大于 6 位；账号为用户的唯一标识，不可重复，且只能由字母和数字组成，长度在 6-16 位之间；用户类别包括普通用户和管理员两种，普通用户只有浏览和购买权限，管理员有管理全站内容的权限。注册成功后系统必须给用户返回注册成功的信息，系统的注册功能必须在 5 秒内完成。”

6. 分析题 (8 分)

敏捷软件开发方法认为过度的依赖文档会使得软件开发非常“笨重”，难以快速应对变化，它建议适当编写文档，更加重视程序代码的编写，你认为敏捷方法的这一思想是否合理，请给出解释和说明。

敏捷软件开发方法的这一思想是合理的。以下是我的解释和说明：

- 快速应对变化：在当今快速发展的市场环境中，需求经常发生变化。如果过度依赖文档，那么每次需求变更都需要更新文档，这将消耗大量时间，降低开发效率。而敏捷方法通过更加重视程序代码的编写，能够更快地响应需求变更，提高开发效率。
- 减少沟通成本：过度的文档编写可能导致团队成员之间的沟通成本增加。而敏捷方法强调团队协作和面对面沟通，可以减少不必要的文档编写，提高团队沟通效率。
- 提高软件质量：敏捷方法通过持续集成、测试驱动开发等实践，使得开发人员在编写代码的过程中就能够及时发现并修复问题，从而提高软件质量。
- 适应性与灵活性：敏捷方法强调适应性和灵活性，通过短周期的迭代开发，可以快速适应市场变化和用户需求。而过度的文档编写可能会限制这种适应性和灵活性。
- 以客户为中心：敏捷方法强调以客户为中心，关注客户的需求和反馈。通过更加重视程序代码的编写，可以更快地实现客户需求，提高客户满意度。

总之，敏捷方法的这一思想是合理的，因为它能够帮助团队更快地应对变化，提高开发效率，减少沟通成本，提高软件质量，增强适应性和灵活性，以及更好地满足客户需求。当然，这并不意味着完全不编写文档，而是要求在保证软件质量的前提下，适当编写文档，以便团队成员之间进行有效沟通和协作。

7. 分析题 (5 分)

根据软件工程的观点，软件是程序+数据+文档，而不仅仅是程序代码；请结合课程软件开发项目的实践，解释说明为什么软件项目的开发需要文档？（3 分）；请列举出 4

个常见的文档例子（2 分）

- 沟通工具：文档是团队成员之间沟通的重要工具。它可以帮助团队成员理解项目目标、需求、设计决策和开发进度等，确保所有人对项目有共同的理解。
- 知识传递：文档可以作为知识的载体，将项目的关键信息传递给未来的维护者或新加入的团队成员。这有助于减少知识流失，确保项目的可持续性。
- 规划与管理：文档可以帮助项目管理者进行项目的规划和跟踪。例如，通过需求文档、项目计划书和测试计划等，管理者可以更好地监控项目进度和质量。
- 质量保证：文档是质量保证的基础。通过需求分析文档、设计文档和测试文档等，可以确保软件开发过程中的每个阶段都符合预定的标准和要求。
- 合规性和审计：在某些行业，如医疗、金融等，软件项目必须遵守特定的法规和标准。文档不仅有助于展示项目的合规性，还是审计过程中的重要依据。

以下是四个常见的文档例子：

- 需求规格说明书（Software Requirements Specification, SRS）：详细描述了软件产品的功能和约束条件，包括用户需求、系统必须遵循的标准以及界面细节等。
- 设计文档：包括概要设计文档和详细设计文档，它们分别描述了软件的高级结构和具体实现细节，如数据库设计、模块划分、接口设计等。
- 测试计划和测试报告：测试计划定义了测试的范围、方法、资源和时间表，而测试报告记录了测试的结果、发现的问题以及解决的措施。
- 用户手册和操作指南：提供了软件产品的使用说明，帮助用户了解如何安装、配置和使用软件，以及如何进行故障排查。

8. 分析题 (5 分)

请阅读以下程序代码，指出其代码编写不规范之处，并说明原因。回答格式可为：第 **行：存在什么问题。

```
1 private void updateRemoteNode(Node node, Cursor c) {
2     SqlNote sqlnote=new SqlNote(mcontext,c);
3     node.setContentbylocalJSON(sqlnote.getContent());
4     GTaskClient.getInstance().addupdatenode(node);
5     updatereotemeta(node.getgid(),sqlnote);
6     if(sqlnote.isnotetype()) {
7         Task task=(Task) node;
8         TaskList preparentlist=task.getParent();
9         String parentgid2=mnidtogid.get(sqlnote.getParentid());
10        TaskList parentlist2=mgtasklisthashmap.get(parentgid2);
11        if (parentlist1!=parentlist2) {
12            parentlist1.removechildtask(task);
13            parentlist2.addchildTask(task);
14            GTaskClient.getInstance().movetask(task,preparentlist,curparentlist);
15        }
16    }
17    sqlNote.resetlocalmodified();
18    sqlNote.commit(true);
19 }
```

缺少必要的注释，不易理解和阅读

第 7-10 行,12-14 行：缺少缩进

第 2 行：变量命名不规范，sqlnote 是一个类的名称，造成望文生义

第 16 行：多余大括号

《软件工程》考试试卷 2

1. 简答题（6 分）

请结合你阅读小米便签开源软件，介绍你在阅读过程中产生了哪些文档（2 分）？为什么需要这些文档（2 分）？这些文档在后续的软件开发中有何作用（2 分）

在阅读小米便签开源软件的过程中，我可能会产生以下几种文档：

- 需求分析文档：这个文档主要是对小米便签软件的功能需求进行详细的分析和描述，包括用户的需求、系统的需求等。这个文档的存在可以帮助我们更好地理解软件的功能和目标，为后续的开发工作提供指导。
- 设计文档：这个文档主要是对软件的架构设计、模块设计、数据库设计等进行详细的描述。这个文档的存在可以帮助我们更好地理解软件的内部结构和工作原理，为后续的开发工作提供参考。
- 测试文档：这个文档主要是对软件的测试计划、测试用例、测试结果等进行详细的记录。这个文档的存在可以帮助我们更好地理解软件的质量情况，为后续的优化和维护工作提供依据。

这些文档在后续的软件开发中有以下作用：

- 提高开发效率：通过阅读这些文档，我们可以快速地了解软件的需求、设计和测试情况，从而减少不必要的沟通和误解，提高开发效率。
- 保证软件质量：这些文档详细记录了软件的需求、设计和测试情况，可以作为评估软件质量的重要依据，帮助我们发现和修复问题，保证软件的质量。
- 便于后续维护：这些文档详细记录了软件的设计和测试情况，可以为后续的软件维护提供重要的参考信息，帮助我们更好地理解和修改软件，降低维护的难度和成本。

补：实践中产生了开源代码的泛读文档，质量分析文档

2. 简答题（4 分）

测试(Testing)和调试(Debugging)均是软件开发过程中的二项重要工作，请说明这二项工作有何区别（2 分）？二者之间有何关系（2 分）？

测试和调试在软件开发中是两项不同的工作，它们的主要区别体现在目的和方法上。

- 目的：测试的目的是发现软件中的错误，而调试则是定位错误并修改程序以修复这些错误。测试通常有计划且有规律，是有预知的过程；而调试往往是无计划且无规律的，其过程和结果是不可预知的。
 - 方法：测试从一个已知的条件开始，使用预先定义的测试用例和过程，而调试则多从一个未知的条件开始，需要开发人员进行推理和分析来定位问题所在。测试可以事先设计，进度可以度量；调试则不能描述具体过程或持续时间。
- 测试和调试之间存在着紧密的联系。
- 测试通过发现软件的不当行为来揭示问题的存在，而调试则是深入到代码层面去理解并解决问题的过程。
 - 测试经常是由独立的测试团队完成，而调试则必须由了解详细设计的开发人员来完成。

综上所述，测试是为了确保软件质量，通过各种测试方法和用例来发现可能存在的问题；而调试则是在测试发现问题后，由开发者进行的一种更深层次的分析 and 修正过程。

3. 分析题（6分）

在阅读小米便签开源软件过程中，我们强调程序质量和编程风格对于开发高质量的软件的重要性。请结合你的课程实践，解释说明

(1) 遵循编程风格规范可以从哪些方面提高软件系统的质量？（3分）

(2) 请列举4项常用的编程风格（3分）

遵循编程风格规范对于提高软件系统的质量至关重要。以下是一些方面：

- 可读性：良好的编程风格可以使代码更易于阅读和理解，降低维护成本。例如，使用一致的缩进、命名规则和注释可以提高代码的可读性。
- 可维护性：遵循编程风格规范有助于保持代码结构的一致性，从而降低维护难度。例如，将逻辑相关的代码放在一起、避免过长的函数和方法等。
- 可扩展性：良好的编程风格有助于提高代码的可扩展性。例如，使用模块化的设计、合理的抽象层次和接口设计等。
- 减少错误：遵循编程风格规范可以减少潜在的错误和漏洞。例如，避免使用容易混淆的变量名、正确使用异常处理等。

常用的编程风格有：

- 驼峰命名法（CamelCase）：用于标识符的命名，例如 myVariable。
- 蛇形命名法（snake_case）：用于文件名、命令行参数等，例如 my_variable。
- 下划线命名法（underscore_case）：用于常量名，例如 MY_CONSTANT。
- 缩进风格：例如使用空格或制表符进行缩进，以及缩进的宽度（通常是4个或8个字符）。

总的来说，编码风格好的四方面：注释，命名，布局，结构

程序质量保证方法

- 遵循编码风格
- 采用程序设计方法
- 开展代码重用
- 进行结对编程

编写代码的基本原则

□ 易读, 一看就懂

- ✓ 理解代码的内涵和意图
- ✓ 望文生义的符号、缩进和括号、代码注释, 遵循编码风格

□ 简明, 减低复杂度

- ✓ 避繁就简、不用goto语句、减少嵌套层数、简单算法

□ 易改, 便于维护

- ✓ 便于修改程序代码、增加新的代码
- ✓ 封装、参数化、模块化、隐藏、常元

□ 无二义, 不产生歧义

- ✓ 不要让人产生误解

编码风格-代码布局

- 缩进, 用好Tab键
- 用括号来表示优先级
- 断行处的{ }
 - if (condition) {
 DoSomething();
} else {
 DoSomething();
}
- 一行至多只一条语句
 - ✓ 不要在一行定义多个变量

```
146 public void onDismiss(DialogInterface dialog) {  
147     stopAlarmSound();  
148     finish();  
149 }  
150  
151 private void stopAlarmSound() {  
152     if (mPlayer != null) {  
153         mPlayer.stop();  
154         mPlayer.release();  
155         mPlayer = null;  
156     }  
157 }  
158 }
```

编码风格-代码组织

- 按一定次序来说明数据
- 按字母顺序说明对象
- 尽可能避免使用嵌套结构
- 采用统一的缩进规则
- 单入口单出口

```
91 private void playAlarmSound() {  
92     Uri url = RingtoneManager.getActualDefaultRingtoneUri(this,  
93     RingtoneManager.TYPE_ALARM);  
94  
95     int silentModeStreams = Settings.System.getInt(getContentResolver(),  
96     Settings.System.MODE_RINGER_STREAMS_AFFECTED, 0);  
97  
98     if ((silentModeStreams & (1 << AudioManager.STREAM_ALARM)) != 0) {  
99         mPlayer.setAudioStreamType(silentModeStreams);  
100     } else {  
101         mPlayer.setAudioStreamType(AudioManager.STREAM_ALARM);  
102     }  
103     try {  
104         mPlayer.setDataSource(this, url);  
105         mPlayer.prepare();  
106         mPlayer.setLooping(true);  
107         mPlayer.start();  
108     } catch (IllegalArgumentException e) {  
109         // TODO: Auto-generated catch block  
110     }  
111 }
```

编码风格-命名规范

- 采用有意义、一目了然的命名方式

- ✓ 变/常/函数/方法/类/包
- ✓ 一看就懂, 望文生义

- 无意义的命名

- ✓ i, j, x, y

- 有意义的命名

- ✓ szPath
- ✓ vPrintName()
- ✓ bReturn

```
38 public class NotesProvider extends ContentProvider {  
39     private static final UriMatcher mMatcher;  
40  
41     private NotesDatabaseHelper mHelper;  
42  
43     private static final String TAG = "NotesProvider";  
44  
45     private static final int URI_NOTE = 1;  
46     private static final int URI_NOTE_ITEM = 2;  
47     private static final int URI_DATA = 3;  
48     private static final int URI_DATA_ITEM = 4;  
49  
50     private static final int URI_SEARCH = 5;  
51     private static final int URI_SEARCH_SUGGEST = 6;  
52 }
```


编码风格-命名

命名通则

- ✓使用英文单词或缩写, 不要使用拼音
- ✓望文知义原则, 含义清晰、明确
- ✓命名不要过长
- ✓尽量使用全称, 少用缩写

不规范的命名

- ✓DaYinWenJian
与 PrintFile

```
public class LoginManager {  
    private UserLibrary userLib;  
  
    public void LoginManager(); //构造函数  
    public int login(account, password); //用户登录  
    public boolean isValid(String account, String password); //判断用户是否合法  
}
```

编码风格-命名

命名和大小写

- ✓类/类型/变量: 用名词和名词短语, 所有单词第一个字母大写
- ✓函数/方法: 动词或者动词短语, 第一个单词小写, 随后单词第一个字母大写
- ✓示例: Member, ProductInfo, getName(), setName(), renderPage()

不规范的命名

- ✓变量: print;
- ✓方法: Print()

```
public class LoginManager {  
    private UserLibrary userLib;  
  
    public void LoginManager(); //构造函数  
    public int login(account, password); //用户登录  
    public boolean isValid(String account, String password); //判断用户是否合法  
}
```

编码风格-代码注释

帮助理解程序

- ✓注释要说明程序: (1) 做什么; (2) 为什么这么做; (3) 注意事项
- ✓无需解释程序如何做

注解位置

- ✓类头、函数/函数头、关键语句块头、关键语句尾

有效、必要、简洁的注释

- ✓太少和过多均不可取
- ✓注释要可理解、准确、无二义

随代码的修改而修改

```
1  /**  
2  * Copyright (c) 2010-2011, The #iCode Open Source Community (www.micode.net)  
3  *  
4  * Licensed under the Apache License, Version 2.0 (the "License");  
5  * you may not use this file except in compliance with the License.  
6  * You may obtain a copy of the License at  
7  *  
8  * http://www.apache.org/licenses/LICENSE-2.0  
9  *  
10 * Unless required by applicable law or agreed to in writing, software  
11 * distributed under the License is distributed on an "AS IS" BASIS,  
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
13 * See the License for the specific language governing permissions and  
14 * limitations under the License.  
15 */  
16  
17 package net.micode.notes.tool;  
18  
19 import android.content.Context;  
20  
21  
22 public class BackupUtils {  
23     private static final String TAG = "BackupUtils";  
24     // Singleton stuff  
25     private static BackupUtils sInstance;  
26  
27     public static synchronized BackupUtils getInstance(Context context) {  
28         if (sInstance == null) {  
29             sInstance = new BackupUtils(context);  
30         }  
31         return sInstance;  
32     }  
33  
34     /**  
35     * Following states are signs to represents backup or restore  
36     * status  
37     */  
38     // Currently, the sdcard is not mounted  
39     public static final int STATE_SD_CARD_UNMOUNTED = 0;  
40 }
```

4. 分析题 (6 分)

请结合针对小米便签软件代码的阅读和维护，说明你们小组是如何进行/开展结对编程的（3 分），这种编程方式有何好处（3 分）

如何实现结对编程

□ 职责明确

- ✓ 一人写设计文档、编写程序和单元测试等等
- ✓ 一人审阅文档、复审程序代码、考虑单元测试的覆盖率、是否需要重构、解决具体的技术问题等等

□ 互换角色

- ✓ 不要连续超过工作1小时，提高效率

□ 主动参与

- ✓ 开展讨论、解决问题、做出贡献

结对编程带来的好处

□ 提高程序质量

- ✓ 提供更好的设计质量和代码质量
- ✓ 合作解决问题能力强, $1+1 > 2$

□ 提升开发效率

- ✓ 开发人员更加信心
- ✓ 有效地避免了闭门造车
- ✓ 更易于发现问题和纠正问题

□ 促进学习交流

- ✓ 有效的学习, 做中学效果更好
- ✓ 相互学习和分享经验
- ✓ 更好应对人员流动, 一个走了另一个人可以替换上

结对编程可以获得更高的投入/产出比

5. 分析题 (6 分)

请你分析为什么敏捷软件开发方法可以有效的应对软件需求的变化（3 分）？该方法采用哪些举措来应对软件需求的变化（3 分）？

敏捷软件开发方法可以有效应对软件需求的变化，主要原因在于其灵活性和迭代性。敏捷开发强调快速响应变化，通过频繁的迭代和持续的反馈，能够及时调整开发方向，以适应不断变化的需求。此外，敏捷开发鼓励团队协作和跨功能团队的建立，有助于更好地理解需求并提供有效的解决方案。

以下是一些敏捷软件开发方法采用的举措来应对软件需求的变化：

- 迭代开发：敏捷开发将项目划分为多个小的迭代周期，每个周期都会产生可交付的产品。这样，团队可以快速看到产品的进展，并根据反馈进行调整。
- 持续集成：敏捷开发鼓励团队成员频繁地集成代码，以便及时发现和解决问题。这有助于减少集成问题，提高软件质量。

- 用户故事和用例：敏捷开发使用用户故事和用例来描述需求，这些描述通常是简短、具体的。这使得团队更容易理解需求，并根据需要进行调整。
- 适应性计划：敏捷开发不依赖于详细的前期计划，而是根据项目进展和需求变化进行适应性计划。这有助于团队灵活应对变化，避免过度设计和浪费资源。
- 测试驱动开发（TDD）：敏捷开发鼓励先编写测试用例，然后再编写实现代码。这有助于确保代码满足需求，并能够在需求变化时快速进行调整。
- 持续交付：敏捷开发追求持续交付可工作的软件，这使得团队能够快速响应需求变化，并及时提供解决方案。

8. 综合分析题 (6 分)

软件需求事先难以确定且在开发过程中会经常性的发生变化，这是软件项目的特点。请针对这一特点，分析一下问题： 1) 软件需求变化会对软件开发产生什么样的影响；（3 分） 2) 如果在开发过程中（如在软件设计阶段）软件需求发生了变化，该如何进行处理？（3 分）

（1）软件需求变化会对软件开发产生以下影响：

- 增加开发成本：需求的变更可能导致已完成的设计、编码等需要重新进行，增加了开发的时间和资源消耗。
- 影响项目进度：频繁的需求变更可能导致项目进度延误，甚至可能无法按时交付。
- 降低软件质量：如果对需求变更处理不当，可能导致软件质量下降，出现功能不完善、性能问题等。
- 增加团队压力：需求变更可能导致团队需要频繁调整工作计划和任务分配，增加了团队成员的工作压力。

（2）如果在开发过程中（如在软件设计阶段）软件需求发生了变化，可以采取以下方式进行处理：

- 评估变更影响：首先评估需求变更的影响范围和程度，包括对已有设计、代码、测试等方面的影响。
- 更新需求文档：将变更后的需求整理成文档，并及时通知相关团队成员，确保大家对新需求有清晰的认识。
- 调整项目计划：根据需求变更的情况，重新制定或调整项目计划，包括时间、资源、任务分配等。
- 修改设计和代码：根据新需求对已有的设计和代码进行修改，确保满足新的功能和性能要求。
- 重新测试：针对变更后的需求进行测试，确保软件的功能和性能符合预期。
- 持续沟通：在整个过程中保持与团队成员的沟通，确保大家对需求变更有共同的理解和认识。

《软件工程》考试试卷 3

1. 简答题 (6 分)

制定软件项目计划要考虑以下三方面的要素：(1) 软件开发过程；(2) 要完成的工作；(3) 约束和限制。请逐一解释为什么要考虑这些因素。

- 软件开发过程：考虑软件开发过程是为了确保项目按照一定的流程和方法进行，从而提高软件开发的效率和质量。一个好的软件开发过程可以帮助团队更好地协作，降低沟通成本，减少错误和遗漏。同时，遵循一个成熟的软件开发过程也有助于提高项目的可预测性和可控性，降低项目风险。
- 要完成的工作：明确要完成的工作是为了确保项目的目标清晰、具体，使团队成员对项目有一个共同的认识。这包括需求分析、系统设计、编码、测试等各个阶段的具体任务。通过明确要完成的工作，可以更好地分配资源，制定合理的时间表，确保项目按时按质完成。
- 约束和限制：考虑约束和限制是为了确保项目在现实条件下能够顺利进行。这些约束和限制可能包括时间、预算、人力资源、技术等方面。在制定项目计划时，需要充分考虑这些因素，合理分配资源，制定可行的方案。同时，了解约束和限制也有助于团队成员在遇到问题时能够迅速调整策略，确保项目能够顺利进行。

2. 分析题 (6 分)

请解释 (1) 迭代软件开发过程模型与增量软件开发过程模型二者之间有何区别；(2) 软件开发过程模型与软件项目计划二者之间有何区别？

软件开发模型：P96，软件项目计划：P482

迭代软件开发过程模型与增量软件开发过程模型的主要区别在于开发方式和反馈机制的不同。

- 迭代模型：是一种重复和改进的开发过程，它强调在每个迭代周期内对产品进行改进和完善，直到最终达到满意的状态。迭代模型允许团队在开发过程中不断地评估和调整，以适应变化的需求和条件。
- 增量模型：则是将整个软件解决方案分解成多个部分或增量，每个增量构建一部分解决方案，并且每个阶段结束后都有一个可交付的产品。与迭代模型不同，增量模型的每个阶段可能不提供完整的功能，而是逐步构建起整个系统。

软件开发过程模型与软件项目计划的区别主要体现在目的和应用上。

- 软件开发过程模型：是软件生产和活动的一种框架，它定义了软件开发的各个阶段和步骤，以及这些步骤之间的关系。过程模型是对软件开发活动的简化表示，可以是通用的也可以是特定于某个领域的。
- 软件项目计划：则是基于特定的软件开发过程模型来制定的，它包括项目的时间表、资源分配、预算、风险管理等具体细节。项目计划是为了指导项目从开始到结束的所有活动，确保项目目标的实现。

3. 分析题 (8 分)

“易变性”是软件开发的特点，它是指用户针对软件的需求经常会发生变化，甚至在开发的后期，请结合你个人的软件开发实践以及对开源软件的阅读，谈谈以下问题：

- (1) 软件系统的需求为什么具有“易变性”的特点：（2分）
- (2) 分别从技术和管理二个角度分析，软件需求的变化会对软件项目开发产生什么样的影响：（3分）
- (3) 结合对开源软件的阅读，请谈谈 OSChina 程序代码有哪些好的举措来应对软件需求的易变性。（3分）

(1) 软件系统的需求具有“易变性”的特点，主要是由于以下原因：

- 用户需求的不确定性：在软件开发初期，用户可能无法准确描述自己的需求，或者在开发过程中发现新的需求。随着项目的推进，用户对软件的认识逐渐加深，可能会提出新的需求或对原有需求进行调整。
- 市场环境的变化：软件开发过程中，市场环境可能发生较大变化，如竞争对手的产品更新、政策法规的调整等，这些因素可能导致软件需求发生变化以适应市场环境。
- 技术的进步：随着技术的不断发展，可能会出现新的技术方案或工具，使得原有的软件需求不再适用，需要进行调整。

(2) 从技术和管理两个角度分析，软件需求的变化会对软件项目开发产生以下影响：

- 技术角度：
- 代码重构：需求的变更可能导致已有代码不再适用，需要进行重构以适应新的需求。
- 测试用例的调整：需求变更后，原有的测试用例可能不再适用，需要根据新的需求重新编写测试用例。
- 文档的更新：需求变更后，相关文档也需要进行相应的更新，以确保团队成员对新需求有清晰的认识。
- 管理角度：
- 项目进度的调整：需求变更可能导致项目进度受到影响，需要重新评估项目的时间安排。
- 资源的重新分配：需求变更可能导致项目资源（如人力、资金等）需要重新分配。
- 风险管理：需求变更可能带来一定的风险，如项目延期、成本增加等，需要对风险进行识别和管理。

(3) 结合对开源软件的阅读，OSChina 程序代码有以下好的举措来应对软件需求的易变性：

- 模块化设计：通过模块化设计，将系统划分为多个相对独立的模块，便于针对特定模块进行调整和优化，降低需求变更带来的影响。
- 版本控制：使用版本控制系统（如 Git）对代码进行管理，可以方便地追踪需求变更前后的代码差异，便于团队成员了解变更内容并进行协作开发。
- 持续集成与持续交付：通过持续集成和持续交付的实践，可以确保软件在需求变更后能够快速地进行构建、测试和部署，提高开发效率。

5. 分析题 (6分)

请结合本课程的讲授，谈谈“程序设计”和“软件开发”二者之间的异同

程序设计和软件开发是软件工程中的两个核心概念，它们之间既有联系也有区别。具体分析如下：

相同点：

- 目标一致：无论是程序设计还是软件开发，它们的最终目的都是为了解决实际问题，提供有效的解决方案。
- 相互依赖：软件开发过程中包含程序设计，没有好的程序设计就无法完成高质量的软件开发；反之，程序设计也需要在软件开发的框架下进行，以确保设计的程序能够被有效地实现并投入使用。

不同点：

- 性质差异：软件开发是一个全面的过程，它包括需求捕捉、需求分析、设计、实现、测试和维护等多个阶段。而程序设计更侧重于解决问题的具体编程过程，它是软件开发中的一个环节，主要关注如何编写代码来实现特定的功能和算法。
- 内容范围：软件开发的内容更为广泛，它不仅包括技术层面的工作，还涉及到项目管理、团队协作、市场调查等多个方面。程序设计则主要集中在技术层面，如选择合适的数据结构和算法，编写高效、可维护的代码等。
- 岗位分工：在实际工作中，软件开发往往需要多个角色的合作，包括前端开发、后端开发、系统架构师、测试工程师等，而程序设计师通常指的是专注于编程和算法实现的工程师。
- 总的来说，程序设计和软件开发都是软件工程中不可或缺的部分，它们相辅相成，共同推动软件项目的顺利完成。理解它们之间的异同有助于更好地进行软件项目的开发和管理。

《软件工程》考试试卷 4

1. 判断题（每个 1 分共 6 分）

- (1). UML 顺序图通过描述类间的消息传递，进而来刻画类与类之间如何通过交互和协作来实现用例功能。（ ）
- (2). 安全性（如确保隐私和防止非法访问）也是一种软件需求。（ ）
- (3). 软件测试的目的是要纠正代码中的错误。（ ）
- (4). 在不改变软件功能的前提下，对软件的整体设计结构及程序代码进行重构，以提高软件质量，也是一种软件维护形式。（ ）
- (5). 软件文档、程序代码、测试用例等均可作为软件配置项纳入配置管理。（ ）
- (6). 建模语言 UML 是给软件分析人员和设计人员用的，编码人员只会用到程序设计语言如 Java，不会用到 UML。（ ）

2. 简答题（6 分）

请结合小米便签阅读和维护课程实践，解释说明（1）编写代码时为什么要遵循编码规范？（2）谁会关注代码编写的规范性？（3）结合例子说明，小米便签程序代码体现了哪些高质量的软件设计原则。

（1）编写代码时遵循编码规范的原因：

- 提高代码的可读性：统一的编码规范使得代码风格一致，有助于团队成员之间的理解和交流，减少因代码风格差异带来的阅读障碍。
- 提高代码的可维护性：遵循规范的代码更容易进行修改和维护，因为规范通常包含了如何组织代码、如何命名变量和函数等最佳实践，这些都能降低后期维护的难度。
- 减少错误的发生：编码规范往往包含了一些避免常见错误的建议，如避免使用魔法数字、避免全局变量滥用等，这有助于减少程序中的潜在错误。

（2）关注代码编写规范性的人群：

- 开发者：开发者是编写代码的主体，他们需要遵循编码规范来保证代码质量，同时也需要在团队内部推广和执行这些规范。
- 代码审查者：在进行代码审查时，审查者会关注代码是否遵循了规范，这有助于发现潜在的问题和改进点。
- 项目经理和团队领导：他们需要确保团队遵循编码规范，以维持代码质量和项目的可持续性。

（3）结合例子说明，小米便签程序代码体现的高质量软件设计原则：

- 模块化设计：小米便签程序代码将功能划分为多个模块，每个模块负责特定的功能，这有助于提高代码的可维护性和可扩展性。
- 单一职责原则：每个模块或类都只负责一项任务，这样可以避免一个模块或类的变动影响到其他部分，降低耦合度。
- 开放封闭原则：小米便签程序代码在设计时考虑到了未来可能的变化，通过抽象和接口的设计，使得系统可以在不修改原有代码的情况下进行扩展。

- 依赖倒置原则：在小米便签程序代码中，高层模块不依赖于低层模块的具体实现，而是依赖于抽象接口，这使得系统更加灵活和稳定。

3. 简答题（6 分）

在软件开发过程中如果软件需求发生了变化，会对该软件系统的哪些软件制品产生影响？结合小米便签开源软件的维护，解释说明你们如何根据变化的软件需求来开展软件开发工作的？（对实现增加的功能进行需求分析，在已有功能的基础上思考如何让用户更加满意）

在软件开发过程中，如果软件需求发生了变化，可能会对以下软件制品产生影响：

- 需求文档：需求变更后，需求文档需要进行相应的更新，以反映新的需求。
- 设计文档：需求变更可能会影响到系统的设计，因此设计文档也需要进行相应的调整。
- 代码：需求的变更可能会导致代码的重构或新增功能的开发。
- 测试用例和测试计划：需求变更后，原有的测试用例可能不再适用，需要根据新的需求重新编写测试用例和测试计划。
- 用户手册和帮助文档：需求变更后，用户手册和帮助文档也需要进行相应的更新，以便用户能够正确理解和使用新的功能。

结合小米便签开源软件的维护，我们可以根据变化的软件需求来开展软件开发工作的方式如下：

- 首先，我们会评估需求变更的影响范围和程度。这包括了解变更的原因、变更的内容以及对现有系统的影响。
- 然后，我们会更新需求文档，将新的需求详细地记录下来。同时，我们也会与团队成员进行沟通，确保大家对新的需求有清晰的认识。
- 接下来，我们会根据新的需求重新评估系统设计，并更新设计文档。这可能涉及到架构的调整、模块的划分等。
- 在代码层面，我们会根据新的需求进行相应的开发工作。这可能包括新增功能的开发、代码的重构等。在这个过程中，我们会遵循编码规范，确保代码的质量。
- 同时，我们会根据新的需求编写测试用例和测试计划，并进行相应的测试工作。这有助于确保新的需求得到了正确的实现，并且系统的稳定性和可靠性不会受到影响。
- 最后，我们会更新用户手册和帮助文档，以便用户能够正确理解和使用新的功能

在整个过程中，我们会密切关注需求变更带来的影响，并与团队成员保持密切的沟通和合作。这有助于确保软件开发工作能够顺利进行，同时也能够应对可能出现的问题和挑战。

4. 简答题（4 分）

软件迭代过程模型和敏捷开发方法是否都能有效应对软件需求的变化，如果不行说明原因，如果可以解释二者在应对变化方面的差别？

软件迭代过程模型和敏捷开发方法都能有效应对软件需求的变化。它们都是为了解决传统瀑布模型在应对需求变化方面的不足而提出的，更注重灵活性和适应性。

- 软件迭代过程模型：它将整个软件开发过程划分为多个较小的迭代周期，每个周期都包括需求分析、设计、编码、测试等阶段。每次迭代都是在前一次的基础上进行改进，逐步完善软件功能。这种模型允许在每个迭代结束时重新评估需求，并根据评估结果调整后续的开发计划。因此，它能够在一定程度上应对需求的变化。
- 敏捷开发方法：它是一种更为灵活的开发方法，强调快速响应变化。敏捷开发中的每个迭代周期（通常为 2-4 周）都会产生一个可交付的软件版本，这使得团队能够更快地

看到成果，并根据用户的反馈进行调整。与迭代模型相比，敏捷开发更加注重团队协作、客户参与和持续改进，这有助于更好地应对需求的变化。

二者在应对变化方面的差别主要体现在以下几个方面：

1. 迭代周期：敏捷开发的迭代周期通常较短，这使得团队能够更快地响应变化。而迭代过程模型的迭代周期可能较长，因此在应对需求变化方面可能不如敏捷开发灵活。
2. 客户参与：敏捷开发强调客户的全程参与，通过定期的沟通和反馈，确保团队始终了解客户的需求。而迭代过程模型可能没有这么高的客户参与度。
3. 文档和过程：敏捷开发倾向于减少不必要的文档和过程，更注重实际工作的完成。而迭代过程模型可能更注重文档和过程的完整性。
4. 团队协作：敏捷开发强调团队成员之间的紧密协作和自组织，这有助于提高团队应对变化的能力。而迭代过程模型可能对团队协作的要求较低。

总的来说，敏捷开发方法在应对软件需求变化方面具有更大的优势，但这并不意味着迭代过程模型无法应对需求变化。在实际项目中，可以根据项目的特点和团队的实际情况选择合适的开发方法。

5. 简答题 (4 分)

说明面向对象软件开发方法提供了哪些技术手段来支持软件重用？

面向对象软件开发方法提供了多种技术手段来支持软件重用，主要包括封装、继承和多态。

1. 封装：封装是面向对象编程的核心概念之一，它将数据和操作这些数据的方法绑定在一起，形成对象。通过封装，对象的内部实现被隐藏起来，外部只能通过对象提供的接口与之交互。这种机制不仅保护了数据的安全性，也提高了代码的模块性，使得代码块可以在不同项目中重用而无需担心内部实现的影响。
2. 继承：继承是面向对象编程中实现代码重用的重要机制。它允许新创建的类（子类）继承现有类（父类）的属性和方法。子类可以复用父类的代码，同时还可以添加或覆盖父类的方法以实现新的功能。这样，已有的功能可以被多个子类共享，减少了重复代码的编写，提高了代码的重用性。
3. 多态：多态是指同一个操作作用于不同的对象时，可以有不同的解释和不同的行为。在面向对象编程中，多态性允许使用一个共同的接口来执行不同对象的行为，这使得程序可以在运行时动态地决定调用哪个对象的方法。多态性提高了代码的灵活性和可扩展性，同时也促进了代码的重用。

综上所述，面向对象软件开发方法通过封装、继承和多态等技术手段，有效地支持了软件的重用，这些特性使得开发者能够构建出既强大又灵活的系统。在实际的软件开发过程中，这些技术手段被广泛应用于各种软件项目中，以提高开发效率和软件质量。

6. 分析题 (10 分)

以下是某个模块的功能描述及接口设计，请采用等价分类法为该模块的集成测试设计

测试用例，详细说明测试用例设计的步骤以及最终的测试用例结果。

Integer: QueryRemainingSeat(TrainNo, CoachClass), 该模块根据输入的高铁车次（假设只有三个车次，分别是：G80、G21、G1034）和座位类别（商务座、一等座、二等座），查询当日该车次及该座位等级剩余的车票数量。如果输入信息不合法，则返回结果为-1。

测试用例设计的步骤如下：

1. 确定输入参数的等价类。在这个例子中，输入参数有两个，分别是 TrainNo 和 CoachClass。对于 TrainNo，我们可以将其分为三个等价类：G80、G21 和 G1034。对于 CoachClass，我们可以将其分为三个等价类：商务座、一等座和二等座。
2. 根据等价类设计测试用例。我们需要为每个等价类至少设计一个测试用例，同时还要考虑到边界条件和非法输入的情况。

最终的测试用例如下：

测试用例	TrainNo	CoachClass	预期结果
TC1	G80	商务座	剩余车票数量
TC2	G80	一等座	剩余车票数量
TC3	G80	二等座	剩余车票数量
TC4	G21	商务座	剩余车票数量
TC5	G21	一等座	剩余车票数量
TC6	G21	二等座	剩余车票数量
TC7	G1034	商务座	剩余车票数量
TC8	G1034	一等座	剩余车票数量
TC9	G1034	二等座	剩余车票数量
TC10	非法车次	商务座	-1
TC11	非法车次	一等座	-1
TC12	非法车次	二等座	-1
TC13	G80	非法座位类别	-1
TC14	G21	非法座位类别	-1
TC15	G1034	非法座位类别	-1

注意：在实际测试过程中，需要将"剩余车票数量"替换为具体的数字。

《软件工程》考试试卷 5

1. 简答题（6 分）

请结合你的课程实践，列举四类软件开发文档（2 分），解释软件开发为什么需要文档（2 分）？分析软件文档会给软件开发带来哪些负面问题（2 分）

四类软件开发文档：

1. 需求分析文档：包括需求规格说明书、功能需求文档等，用于明确软件的功能和性能要求。
2. 设计文档：包括概要设计说明书、详细设计说明书等，用于描述软件的架构、模块划分和接口设计等。
3. 测试文档：包括测试计划、测试用例、测试报告等，用于记录软件的测试过程和结果。
4. 用户文档：包括用户手册、安装指南等，用于指导用户如何使用和维护软件。

软件开发需要文档的原因：

1. 提高沟通效率：文档可以作为团队成员之间沟通的桥梁，确保信息传递的准确性和一致性。
2. 便于项目管理：通过文档可以对项目进度、资源分配等进行有效管理，确保项目按计划进行。
3. 降低风险：文档可以帮助团队识别潜在问题，提前采取措施降低风险。
4. 便于维护：详细的文档可以帮助后续维护人员快速了解软件的结构和功能，提高维护效率。

软件文档可能带来的负面问题：

1. 编写和维护成本：编写高质量的文档需要投入大量的时间和精力，可能会增加项目的成本。
2. 更新不及时：软件在开发过程中可能会发生很多变更，如果文档更新不及时，可能会导致文档与实际软件不一致，从而产生误导。
3. 过于依赖文档：过分依赖文档可能导致团队成员忽视实际操作和实践经验，影响软件开发的效果。

2. 简答题（4 分）

请结合课程实践，解释和说明 Git 工具在软件开发中起到什么作用（2 分）？它主要提供了哪些功能（2 分）？

Git 在软件开发中起到版本控制和团队协作的作用，它主要提供了版本控制、分支管理、合并代码等功能。

Git 作为一款分布式版本控制系统，它在软件开发中扮演着至关重要的角色。通过使用 Git，开发者可以对项目的每一次更改进行跟踪，确保在任何时候都能回退到之前的版本。这种能力对于管理复杂的开发过程尤为重要，特别是在多人协作的项目中。Git 允许每个开发者在本地拥有完整的版本库，这意味着他们可以在不联网的情况下进行开发，极大地提高了开发效率和灵活性。此外，Git 的分支功能使得开发者可以在不同的特性或修复上并行工作，而这些分支最终可以被干净地合并回主干，这为团队协作提供了极大的便利。

而 Git 的主要功能包括仓库的创建与管理、提交更改、查看历史记录以及回滚到指定版本等基本操作。它还支持创建分支来隔离开发新功能或修复 bugs 的工作，并且能够将这些分支合并回主分支。Git 的分布式特性意味着每个开发者都有一个完整的项目副本，这使得本地开发和版本控制更加便捷。同时，Git 还支持与远程仓库进行交互，例如推送更改和拉取更新，以实现团队之间的同步工作。

综上所述，Git 不仅为个人开发者提供了强大的版本控制工具，而且其分布式特性和高效的分支管理能力也使其成为团队合作中不可或缺的一部分。

简单题 (6 分)

请提出 3 种可以提高软件系统质量的软件工程方法（3 分），并解释他们为什么可以提高软件开发的质量（3 分）？

提高软件系统质量的软件工程方法包括：

1. 测试驱动开发（TDD）：在编写实际代码之前，先编写测试用例。这样可以确保开发人员对预期的功能有清晰的理解，并且可以在开发过程中持续验证代码的正确性。通过这种方法，可以及早发现和修复错误，从而提高软件的整体质量。
2. 持续集成（CI）与持续部署（CD）：通过自动化的构建和测试流程，确保代码的每次提交都能快速地被验证和集成到主干上。这有助于及时发现和修复集成错误，减少手动测试的工作量，提高开发效率和软件质量。
3. 代码审查：在代码合并到主干之前，进行同行评审。通过人工检查代码的质量、可读性和一致性，可以发现潜在的问题和改进点。代码审查有助于提高团队协作，促进知识共享，从而提高软件的整体质量。

这些方法可以提高软件开发质量的原因：

- 测试驱动开发（TDD）能够确保开发人员始终关注客户的需求，通过编写测试用例来明确功能的预期结果，从而减少开发过程中的错误和误解。
- 持续集成（CI）与持续部署（CD）通过自动化的方式，使得代码的构建、测试和部署变得更加高效和可靠。这有助于缩短开发周期，提高软件的交付速度和质量。
- 代码审查是一种有效的质量保证手段，它能够发现代码中的问题和改进点，提高代码的可维护性和可读性。同时，代码审查也有助于团队成员之间的知识共享和技能提升。

分析题 (6 分)

软件开发初期要完整、准确地获取软件需求是非常困难的，甚至是不可能的。在软件开发过程中，软件需求会发生变化。请说明软件需求的变化会对软件开发产生什么样的影响？（3 分）？在这种情况下会对软件设计提出什么样的要求？（3 分）

软件需求的变化会对软件开发产生以下影响：

1. 增加开发成本：需求的变更可能导致已经完成的部分需要重新设计、编码和测试，这会增加额外的工作量和成本。
2. 延长开发周期：需求的变更可能导致项目进度的延迟，因为需要对新的需求进行分析、设计和实现。
3. 影响团队士气：频繁的需求变更可能导致团队成员感到困惑和挫败，影响团队的士气和协作。

4. 降低软件质量：在需求不断变化的情况下，可能没有足够的时间进行充分的测试和质量保证，从而导致软件质量下降。

5. 增加风险：需求变更可能导致一些未知的问题和风险，如兼容性问题、性能问题等。

在这种情况下，软件设计需要满足以下要求：

1. 灵活性：软件设计应该具有一定的灵活性，能够适应需求的变化，减少因需求变更带来的影响。

2. 可扩展性：软件设计应该具有良好的可扩展性，以便在未来可以轻松地添加新功能或修改现有功能。

3. 模块化：软件设计应该采用模块化的方法，将系统划分为多个独立的模块，这样在需求发生变化时，只需要修改相关的模块，而不需要对整个系统进行大的改动。

4. 易于维护：软件设计应该注重代码的可读性和可维护性，以便于后续的维护和升级。

5. 文档完善：软件设计应该有完善的文档，包括需求文档、设计文档和测试文档等，以便于团队成员之间的沟通和协作。

综合分析题 (6 分)

对现有软件开发项目的情况进行统计，可以发现软件维护的形式多样，成本很高，周期很长。请结合这一现象，阐述在软件开发阶段应该采取哪些举措来应对软件维护的上述挑战和问题？

(6)

为了应对软件维护的形式多样、成本高和周期长的挑战，软件开发阶段可以采取以下举措：

1. ****明确需求****：在软件开发初期，与利益相关者进行充分的沟通，确保对软件的需求有清晰、准确的理解。这有助于减少后续因需求不明确或变更导致的维护成本。

2. ****模块化设计****：采用模块化的设计方法，将系统划分为多个独立的模块。这样在维护阶段，只需要关注和修改相关的模块，而不需要对整个系统进行大的改动。

3. ****代码质量****：注重代码的可读性和可维护性，遵循编程规范和最佳实践。高质量的代码可以降低维护的难度和成本。

4. ****文档完善****：编写完善的文档，包括需求文档、设计文档、测试文档等。这些文档可以帮助维护人员快速了解系统的架构和功能，提高维护效率。

5. ****自动化测试****：建立自动化测试框架，确保在每次修改后都能快速地进行回归测试。这有助于及时发现问题，降低维护成本。

6. ****持续集成与持续部署****：引入持续集成（CI）和持续部署（CD）的实践，使得代码的构建、测试和部署变得更加高效和可靠。这有助于缩短维护周期，提高软件的交付速度和质量。

7. ****技术债务管理****：在开发过程中，及时处理技术债务，避免问题的累积。这有助于降低后续维护的难度和成本。

8. ****知识共享与培训****：促进团队成员之间的知识共享和技能提升，确保团队成员具备足够的能力来应对维护挑战。同时，为新加入的维护人员提供培训，帮助他们快速熟悉项目。

9. ****反馈机制****：建立有效的反馈机制，收集用户和客户的反馈意见，及时调整维护策略和计划。这有助于确保软件维护能够满足用户的需求和期望。

10. ****预防性维护****：定期进行预防性维护，检查和修复潜在的问题，避免问题的累积和爆发。这有助于降低维护成本和风险。

综合分析题 (16 分)

以下是一段 12306 火车票购买软件的需求描述，刻画了旅客查询和订购火车票的需求信息，请针对该需求描述（1）绘制出 user case 模型（4 分）；（2）分别绘制出查询火车票和订购火车票的顺序图（8 分）；（3）绘制出针对该软件需求的类图（4 分）。

说明：要求正确使用 UML 的图形化模型以及各个图形符号

“用户输入“出发日期”、“出发时间段”、“出发站”、“到达站”、“火车类别”信息，系统将按照时间先后次序，列举出所有满足条件的火车车次信息，包括车次、出发站、始发站、到达站、目的站、出发时间、到达时间，剩余车票数量等等用户在购买车票之前需要登录系统，随后提供出发的日期、出发站、到达站信息，系统将根据上述信息查询出满足要求的车次，用户从中选择欲购买的车次，确定欲购买车票的数量，系统将告知需要用户支付的价格，并要求用户提供支付火车票的信用卡及其所属的银行；随着连接该银行的支付系统完成支付，支付成功后系统将确认车票购买成功，随后给用户发去成功购票的短信。”

《软件工程》考试试卷 6

1. 简答题（4 分）

说明软件测试与程序调试二者之间的区别（2 分）及联系（2 分）？

软件测试与程序调试在软件开发过程中是两个不同但相互关联的活动。它们的区别主要体现在目的、参与角色和执行阶段上，而联系则在于它们共同的目标是提高软件的质量和确保按时交付。

****区别**：**

1. ****目的不同**：**软件测试的主要目的是发现软件中存在的错误，而程序调试的目的是定位并解决这些错误。测试通常以已知条件开始，使用预先定义的测试用例，期望的结果也是预先设定的，测试人员不知道的是软件是否能够通过这些测试用例。相反，调试通常是从未知的内部条件开始，开发人员需要找出导致软件行为异常的具体原因，并修复它。
2. ****参与角色不同**：**软件测试可以由专门的测试人员来完成，尤其是在黑盒测试中，而单元测试和集成测试则可能由开发人员自己来执行。程序调试则主要由开发人员来完成，因为他们对代码的理解最深，能够有效地找到问题并解决它。
3. ****执行阶段不同**：**软件测试贯穿整个软件开发生命周期，从需求分析、设计到编码、维护阶段都有测试活动。而程序调试主要发生在开发阶段，当开发人员编写或修改代码后，他们需要调试以确保代码的正确性。

****联系**：**

尽管软件测试和程序调试在目的等方面有所不同，但它们的最终目标是一致的，即确保软件的质量和按时交付。为了达到这一目标，测试人员和调试人员需要相互尊重、密切配合。测试人员通过测试发现的缺陷需要调试人员去定位和修复，而调试人员修复后的代码又需要测试人员重新验证其正确性和稳定性。这样的互动确保了软件在发布时能够满足用户的需求和预期。

总结来说，软件测试和程序调试虽然在执行细节和侧重点上有所不同，但它们都是软件开发过程中不可或缺的环节，共同作用于提升软件产品的整体质量。

2. 简答题（4 分）

“软件工程”包含哪些基本要素（2 分）？这些要素如何支撑软件系统的工程化开发（2 分）？

软件工程的基本要素包括：方法、工具和过程。这些要素相互关联，共同支撑软件系统的工程化开发。

方法：软件工程中的方法是一组最佳实践的集合，用于解决软件开发中的特定问题。例如，面向对象方法、结构化方法和敏捷方法等。每种方法都有其独特的原理、原则和技术，可以指导开发人员更好地进行软件设计和开发。

工具：软件工程的工具是支持开发过程的软件和硬件工具。这些工具可以帮助开发人员更高效地完成各种任务，例如需求分析、设计、编码、测试和部署等。工具可以包括开发环境、版本控制工具、测试工具、自动化工具等。

过程：软件工程的过程是一组用于管理软件开发和维护的框架。它定义了软件开发过程中的各种活动和任务，以及这些活动和任务之间的关系。过程可以包括需求工程、设计过程、编码过程、测试过程和维护过程等。通过采用标准化和量化的过程，可以确保软件开发的可靠性和效率。

在软件系统的工程化开发中，这些要素的作用如下：

方法：通过采用适合的方法，可以更好地指导开发人员解决软件开发中的问题。这有助于确保软件开发的正确性和有效性。

工具：通过使用适合的工具，可以提高开发效率和质量。这可以减少开发时间和成本，并提高软件产品的质量和可靠性。

过程：通过采用标准化的过程，可以更好地管理软件开发和维护。这有助于确保软件开发的规范化和一致性，提高软件产品的可维护性和可扩展性。

综上所述，软件工程的基本要素和方法、工具、过程相互关联，共同支撑软件系统的工程化开发。通过采用适合的方法和工具，以及标准化的过程，可以更好地实现软件开发的可靠性和效率。

3. 简答题（6 分）

结合小米便签开源代码的阅读和维护，列举出小米便签代码质量有哪些好的方面（4 分）；存在哪些方面的不足（2 分）

小米便签的代码质量在以下方面表现良好：

1. ****代码结构清晰**：**小米便签的代码结构设计合理，模块划分明确，便于阅读和维护。
2. ****命名规范**：**变量、函数和类的命名都遵循了一定的规范，使得代码易于理解。
3. ****注释详细**：**关键部分的代码都有详细的注释，方便其他开发者快速理解代码的功能和实现方式。
4. ****错误处理完善**：**代码中对可能出现的错误进行了充分的考虑和处理，提高了软件的稳定性。
5. ****测试用例丰富**：**项目包含了丰富的测试用例，有助于确保代码的正确性和稳定性。

然而，小米便签的代码也存在一些不足之处：

1. ****代码冗余**：**部分代码存在冗余，可能导致维护成本增加和潜在的 bug。
2. ****部分设计不够优雅**：**某些设计可能不是最优解，可能需要进一步优化以提高代码的可读性和可维护性。

以上分析基于对小米便签开源代码的阅读和维护经验，不同人可能会有不同的观点。

4. 简答题（6 分）

根据软件工程的思想，软件设计通常需要遵循以下的原则：模块化、分解、信息隐藏，从而提高软件系统的质量。请解释为什么这三项原则有助于提升软件设计质量（每个 2 分）

模块化、分解和信息隐藏是软件设计中的重要原则，它们有助于提升软件设计质量的原因如下：

1. ****模块化****：模块化是指将软件系统划分为多个模块，每个模块负责一部分功能。模块化可以提高软件的可读性和可维护性，因为每个模块都可以独立开发、测试和维护。此外，模块化还有助于提高软件的可重用性，因为某个模块可以在其他项目中重复使用。
2. ****分解****：分解是将复杂的问题或任务分解为更小、更易于管理的部分。通过分解，可以将复杂的软件系统划分为多个子系统或组件，每个子系统或组件负责一部分功能。这样可以减少单个模块的复杂性，使得开发人员更容易理解和处理。同时，分解也有助于提高软件的可扩展性，因为新的功能可以通过添加新的子系统或组件来实现，而不需要修改现有的代码。
3. ****信息隐藏****：信息隐藏是指将某个模块的内部实现细节隐藏起来，只暴露必要的接口给其他模块。这样可以降低模块之间的耦合度，使得模块更加独立。当需要修改某个模块的内部实现时，不会影响到其他模块，从而降低了修改的风险和成本。此外，信息隐藏还有助于提高软件的安全性，因为外部模块无法直接访问内部数据，从而减少了数据被恶意篡改的风险。

综上所述，模块化、分解和信息隐藏这三项原则通过简化复杂性、降低耦合度、提高独立性等途径，有助于提升软件设计的质量。

5. 分析题（4分）

请解释说明软件维护和软件开发二者之间的区别和联系（4分）

软件维护和软件开发是软件生命周期中的两个不同阶段，它们之间既有区别也有联系。

****区别****：

- ****目的不同****：软件开发主要是创造新的软件产品，满足用户的需求；而软件维护则是确保已有软件的正常运行和持续发展，包括对软件进行修改、更新、修复和优化等活动。
- ****活动内容不同****：软件开发包括需求分析、设计、编码、测试等一系列从零开始的活动；软件维护则更多是对现有软件的改进和修正，可能包括修复 bug、添加新功能、提升性能等。
- ****成本结构不同****：软件开发的成本主要集中在人力和资源的投入上；而软件维护的成本则可能包括运行成本、维护人员的培训成本以及由于维护活动导致的其他问题的成本。

****联系****：

- ****生命周期的连续性****：软件维护是软件开发生命周期的后续阶段，软件开发完成后进入维护阶段，两者共同构成了软件的完整生命周期。
- ****可维护性的影响****：软件开发阶段的设计和实现直接影响到软件的可维护性，良好的设计和文档化可以大大降低后期维护的难度和成本。
- ****重用的可能性****：在软件维护过程中，可能会重用软件开发阶段的成果，如代码、组件或模块，以提高维护效率和降低总成本。

总结来说，软件维护和软件开发虽然在目标、活动内容和成本结构上有所不同，但它们都是软件生命周期中不可或缺的组成部分，且相互影响。软件开发的质量直接关系到后续维护的难易程度，而有效的软件维护则能延长软件的生命周期，保证其持续满足用户需求。

6. 分析题 (6 分)

软件测试包括哪些工作 (2 分)? 为什么测试用例的设计对于软件测试而言非常重要

(2 分)? 请解释软件测试的原理 (2 分)

书上 P409

包括: 指定软件测试计划, 设计软件测试用例, 执行软件测试活动, 回归测试

为什么测试用例的设计对于软件测试而言非常重要?

1. 提高准确性: 通过设计详细的测试用例, 可以确保软件功能按照预期运行, 并捕获潜在的错误。
2. 提升效率: 标准化和可重复的测试用例使测试过程更加高效, 从而提高整个软件开发的效率。
3. 降低风险: 有效的测试用例有助于发现和解决问题, 从而减少软件发布前的风险。
4. 促进沟通: 测试用例可作为开发人员、测试人员和其他利益相关者之间的沟通依据, 确保所有人对软件有共同的理解。
5. 提升生产力: 在设计阶段发现并修复问题, 可以节省后期大量的时间和资源, 提高软件开发的总体生产力。

软件测试的原理可以理解为: 通过一定的技术和方法, 对软件产品进行功能、性能和安全等方面的检测, 确保软件质量符合要求。其核心目的是发现问题、跟踪缺陷并及时修复, 从而保证软件的质量和稳定性。

8. 综合分析题 (8 分)

为什么要将软件测试分为单元测试、集成测试和确认测试等不同的阶段 (2 分)? 这些阶段通常分别由什么样的人员来完成 (2 分)? 针对这些阶段的测试用例分别在软件开发的哪些阶段来生成 (2 分)? 为什么? (2 分)

将软件测试分为单元测试、集成测试和确认测试等不同阶段的主要原因如下:

1. 降低测试难度: 将软件测试按照开发阶段进行划分, 可以使得测试工作更加集中和有针对性。每个阶段的测试都关注不同的方面, 从而降低了测试的难度和复杂性。
2. 提高测试效率: 按照开发阶段进行测试, 可以使得测试人员更加聚焦于某个阶段的测试任务, 避免浪费时间和资源。同时, 每个阶段的测试结果可以作为下一阶段测试的参考, 提高了测试效率。
3. 尽早发现缺陷: 单元测试是集成测试的基础, 如果在单元测试阶段发现并修复缺陷, 可以避免这些缺陷在后续的测试阶段被再次发现。同时, 尽早发现缺陷也有助于提高开发效率和软件质量。

这些阶段通常由以下人员来完成: (看书 P420)

1. 单元测试：通常由开发人员完成。开发人员在编写代码的同时，需要编写单元测试用例，对自己的代码进行测试。
2. 集成测试：通常由测试人员完成。测试人员在单元测试的基础上，将各个模块集成在一起进行测试，检查模块之间的交互和数据流是否符合预期。
3. 确认测试：通常由专业的软件测试人员完成。在软件开发完成后，测试人员需要对软件进行全面的测试，确保软件功能、性能 and 安全性等方面符合要求。

针对这些阶段的测试用例通常在以下阶段生成：

1. 单元测试用例：在需求分析和设计阶段生成。在这个阶段，开发人员需要对每个模块的功能和接口进行分析和设计，并基于这些需求编写单元测试用例。
2. 集成测试用例：在设计和编码阶段生成。在这个阶段，开发人员已经完成了各个模块的开发工作，测试人员需要根据模块之间的交互和数据流编写集成测试用例。
3. 确认测试用例：在需求分析和设计阶段生成。在这个阶段，测试人员需要根据用户需求和系统要求，编写确认测试用例，对软件的全面功能、性能 and 安全性等方面进行测试。

这些阶段的划分和由哪些人完成以及针对这些阶段的测试用例的生成和在哪些阶段生成的原因主要是为了提高软件质量、降低软件缺陷和提高软件开发的效率。通过将软件测试划分为不同的阶段，可以使得每个阶段的测试更加集中和有针对性，尽早发现并修复缺陷，避免在软件开发后期才发现问题而导致大量的修改和返工。同时，每个阶段的测试都有相应的责任人和任务分工，可以提高软件开发的协作性和效率。

《软件工程》考试试卷 7

1. 简答题（6 分）

结合阅读和标注小米便签开源软件实践，说明小米便签代码中有哪些“好”的软件开发实践和经验值得你学习？（3 分），小米便签代码还有哪些“不好”的质量表现？（3 分）

小米便签代码中“好”的软件开发实践和经验包括：

1. ****模块化设计****：代码结构清晰，模块划分明确。
2. ****命名规范****：变量、函数和类的命名遵循了一定的规范，使得代码易于理解。
3. ****注释详细****：关键部分的代码都有详细的注释。
4. ****错误处理完善****：对可能出现的错误进行了充分的考虑和处理。
5. ****测试用例丰富****：项目包含了丰富的测试用例，有助于确保代码的正确性和稳定性。

然而，小米便签代码也存在一些“不好”的质量表现：

1. ****代码冗余****：部分代码存在冗余，可能导致维护成本增加和潜在的 bug。
2. ****部分设计不够优雅****：某些设计可能不是最优解，可能需要进一步优化以提高代码的可读性和可维护性。

以上分析基于对小米便签开源代码的阅读和实践经验，不同人可能会有不同的观点。

2. 简答题（6 分）

为什么软件系统的需求容易发生变化？（3 分），请分析软件需求的变化会对软件开发活动和软件制品各产生什么样的影响（3 分）？

软件系统的需求容易发生变化的原因包括：

1. ****市场环境变化****：随着市场环境的变化，用户的需求和期望也在不断变化。
2. ****技术进步****：新的技术和工具的出现可能会引发新的需求。
3. ****竞争压力****：为了保持竞争力，企业需要不断地更新和优化其软件产品。
4. ****法规政策变动****：法律法规的变更可能迫使企业调整其软件系统以符合新的标准。
5. ****用户需求变化****：用户在使用软件的过程中可能会产生新的需求或对现有功能提出改进意见。

需求的变化对软件开发活动的影响包括：

1. ****开发计划调整****：需求的变化可能导致项目的开发计划、预算和时间表需要进行调整。
2. ****设计和实现变动****：需求的变更可能需要对软件的设计和实现进行相应的修改。
3. ****测试用例更新****：需求的变化意味着需要更新测试用例以确保新的功能被正确实现并满足需求。
4. ****文档修订****：需求变更后，相关的技术文档和用户手册等也需要进行相应的更新。

需求的变化对软件制品的影响包括：

1. ****功能增加或修改****：需求的变化可能导致软件的功能增加或修改。
2. ****性能要求提高****：新的需求可能对软件的性能提出了更高的要求。
3. ****用户体验改变****：需求的变更可能影响用户的使用体验，例如界面布局、操作流程等。
4. ****稳定性和安全性问题****：需求的变更可能引入新的稳定性和安全性问题。

因此，对于软件系统的需求变更，需要有一套完善的变更管理流程和机制，以确保需求变更能够得到有效控制和管理，减少对软件开发活动和软件制品的负面影响。

3. 分析题（6分）

请结合课程实践，说明在面向对象需求分析阶段，需求模型中“用例图”、“顺序图”、“分析类图”三者间保持一致性是指什么含义？（3分）

在面向对象需求分析阶段，需求模型中的“用例图”、“顺序图”和“分析类图”三者间保持一致性是指：

1. ****用例图****：描述了系统与外部交互者（如用户或其他系统）之间的交互。它展示了系统的功能需求，即系统应该做什么。
2. ****顺序图****：是一种交互图，展示了对象之间是如何协作完成特定用例的。它详细描述了消息的顺序和时间关系。
3. ****分析类图****：描述了系统中的类及其属性、操作和相互关系。它展示了系统的静态结构。

这三者间的一致性意味着它们共同描述了同一个系统的需求和功能，且没有矛盾或冲突。具体来说，用例图中的每个用例都应该能够通过顺序图和分析类图来详细描述其实现过程和涉及的类。而顺序图和分析类图也应该能够准确地反映用例图中描述的功能需求。

4. 简单题（6分）

在软件需求分析阶段，质量保证对于成功地开发软件系统至关重要。请列举在需求分析阶段能够有效确保需求分析及其制品质量的举措有哪些？（3分），并解释原因（3分）。

在软件需求分析阶段，能够有效确保需求分析及其制品质量的举措包括：

1. ****明确需求****：与项目干系人进行深入的沟通，确保对需求有清晰的理解。
2. ****使用标准化的需求描述语言****：如使用一致的术语和格式来描述需求，避免歧义。
3. ****创建详细的需求文档****：记录所有的需求细节，并确保文档易于理解和更新。
4. ****进行需求验证和确认****：通过审查和测试来验证需求的完整性、一致性、可实施性和可测试性。
5. ****建立变更控制机制****：当需求发生变化时，有一个明确的流程来管理这些变更。
6. ****持续跟踪和管理需求****：确保需求的追踪性，以便在后续开发过程中能够对应到相应的功能。

这些举措之所以能够确保需求分析及其制品的质量，原因包括：

- 明确需求有助于减少误解和假设，确保所有人对需求有共同的理解。
- 使用标准化的需求描述语言可以避免歧义，提高沟通效率。

- 详细的需求文档为后续的开发和测试提供了明确的指导，有助于减少错误和遗漏。
- 需求验证和确认可以及早发现潜在的问题和冲突，避免在后期解决这些问题时产生更高的成本。
- 变更控制机制有助于管理需求的变更，防止无序的变更导致混乱和错误。
- 持续跟踪和管理需求有助于确保需求的完整性和一致性，同时也方便了后续的维护和升级。

《软件工程》考试试卷 8

1. 简答题（6 分）

请从软件工程师的角度，解释说明软件需求分析、软件设计、编码实现这三者的开发任务、输出制品有哪些差异性？

软件需求分析、软件设计、编码实现是软件开发过程中的三个关键阶段，每个阶段都有其特定的开发任务和输出制品。

1. ****软件需求分析****:

- ****开发任务****: 在这个阶段，软件工程师需要与客户或利益相关者沟通，了解他们的需求和期望，然后将这些需求转化为详细的软件需求规格。

- ****输出制品****: 需求规格说明书（SRS），它详细描述了软件系统的功能和约束条件。

2. ****软件设计****:

- ****开发任务****: 在设计阶段，软件工程师需要根据需求规格说明书来设计软件的架构和组件。这包括确定系统的模块、接口、数据结构和算法等。

- ****输出制品****: 设计文档，包括高层设计（HLD）和详细设计（LLD）文档。HLD 描述了系统的高级结构，而 LLD 则提供了模块内部的详细设计信息。

3. ****编码实现****:

- ****开发任务****: 在编码阶段，软件工程师将设计文档转化为实际的代码。这涉及到编写、测试和调试代码，以确保它满足设计文档中的要求。

- ****输出制品****: 源代码和相关的注释文档。源代码是实现软件功能的计算机程序，而注释文档则为后续的维护和升级提供了必要的信息。

这三个阶段的主要差异在于它们的关注点和输出制品。需求分析关注的是理解用户的需求，输出的是需求规格说明书；软件设计关注的是如何满足这些需求，输出的是设计文档；编码实现关注的是将设计转化为实际的代码，输出的是源代码和注释文档。

2. 简答题（6 分）

UML 提供了哪些模型可用来对软件系统的行为特征进行描述和建模？

UML（统一建模语言）提供了多种模型来描述和建模软件系统的行为特征，主要包括****用例图、序列图、协作图、状态图和活动图****等。

1. ****用例图****: 从用户的角度描述系统功能，并指明各功能的操作者。它帮助开发人员理解系统的业务需求和目标。

2. ****序列图****: 展示对象之间交互的顺序，即消息发送的时间顺序。它用于描述特定的场景或操作流程。

3. ****协作图****: 与序列图类似，但重点在于对象之间的结构关系，而不是消息的时序。它强调了对象间的组织结构。

4. ****状态图****: 描述一个对象可能处于的不同状态以及触发状态转换的事件。它对于理解系统中复杂对象的行为特别有用。

5. ****活动图****：用来表示工作流或业务流程，展示操作的顺序。活动图可以看作是状态图的一种变体，它更侧重于操作的流程而非状态的变化。

这些模型各自关注系统行为的不同方面，共同构成了对软件系统行为特征的全面描述。在实际应用中，开发人员会根据需要选择合适的模型来表达系统的特定行为，以便更好地理解 and 设计软件系统。

3. 简答题（6 分）

质量对于软件系统而言是非常重要的。请说明在软件开发过程中需要采用哪些方面的工程化手段来保证所开发软件系统的质量。

在软件开发过程中，保证软件系统的质量是非常关键的。为了达到这一目标，需要采用多方面的工程化手段。以下是一些主要的工程化手段：

1. 需求管理：明确、完整、一致的需求是保证软件质量的基础。在软件开发过程中，需要对需求进行详细的梳理、分析和确认，确保开发团队对需求的理解是准确无误的。
2. 软件开发生命周期（SDLC）：选择合适的软件开发生命周期模型（如瀑布模型、迭代模型、敏捷开发等），有助于按照阶段进行质量控制，保证每个阶段输出的正确性。
3. 质量保证（QA）：质量保证是软件开发过程中的一个关键环节，它涉及到代码审查、测试、静态代码分析等手段，目的是发现和修复潜在的问题，从而提高软件的质量。
4. 代码审查：代码审查是一种有效的质量保证手段，通过让同事或其他有经验的开发者检查代码，可以发现潜在的问题和改进点。
5. 单元测试和集成测试：单元测试是针对代码的某个模块进行的测试，而集成测试则是测试多个模块之间的交互。通过这两种测试，可以发现代码中的问题，确保软件在细节和整体上都能正常工作。
6. 持续集成（CI）/持续部署（CD）：这是一种自动化工具，用于自动构建、测试和部署软件。它可以快速发现代码中的问题，减少手动干预，提高软件的质量。
7. 用户反馈和监控：收集用户反馈，监控软件的运行状况，是发现和修复问题的有效手段。通过用户反馈和监控，可以及时发现和修复潜在的问题，提高软件的用户满意度。
8. 文档编写：编写详细的文档，包括用户手册、开发文档等，可以帮助开发团队理解软件的功能和实现方式，同时也可以帮助用户更好地使用软件。
9. 软件开发生态系统（SDAE）：通过建立包含多个利益相关者的开放式生态系统，可以更全面地了解和理解软件需求、问题和挑战，从而更有效地解决问题和改进软件质量。

这些工程化手段可以单独或组合使用，根据项目的实际情况和需要进行选择和应用。通过这些手段的应用，可以提高软件的质量，减少错误和缺陷，从而提高用户的满意度和使用体验。

5. 综合分析题 (25 分)

针对 12306 软件系统，请采用 UML 进行建模和分析。

1) 系统为用户提供用户注册、查询车次、购买车票、退票、改签车次等常用功能

2) 用户购买车票的大体流程是：首先登录到系统之中，查询和选择车次，提供支付

银行账号信息，随后系统将向用户发送短信验证码，用户输入验证码确认正确后，与银行系统进行交互，完成支付，最后完成购票业务，并给用户发送短信以反馈车票信息。

(1) (10 分) 使用 UML 用例图对该系统的需求进行建模。

(2) (10 分) 用 UML 顺序图对用户注册的业务流程进行建模。

(3) (5 分) 基于用例图和顺序图，给出上述系统的 UML 类图。

《软件工程》考试试卷 9、10

1. 简答题（8 分）

请结合课程实践，解释说明软件文档和模型在软件开发过程中发挥了什么作用？

软件文档和模型在软件开发过程中发挥了重要作用，具体如下：

1. **软件文档的作用**：

- **传递信息**：软件文档是团队成员之间沟通的桥梁，它记录了软件开发过程中的需求、设计、实现和测试等方面的详细信息，确保团队成员对项目有共同的理解。
- **便于维护**：良好的文档可以帮助开发人员快速了解系统的结构和功能，便于后续的维护和升级。
- **提高开发效率**：通过查阅文档，开发人员可以快速掌握项目的背景和要求，避免重复工作，提高开发效率。
- **降低风险**：文档可以帮助识别潜在的问题和风险，为项目决策提供依据，降低项目失败的风险。

2. **软件模型的作用**：

- **抽象表示**：模型是对软件系统的一种抽象表示，它可以帮助我们更好地理解系统的结构和行为，为设计和实现提供指导。
- **辅助设计**：通过构建模型，开发人员可以在早期阶段发现设计中的问题和不足，从而优化设计，提高软件质量。
- **促进沟通**：模型是一种直观的表达方式，可以清晰地展示系统的各个方面，有助于团队成员之间的沟通和协作。
- **支持验证**：模型可以用来验证系统的正确性和完整性，确保系统满足需求和约束条件。

1. 简答题（6 分）

软件开发过程中通常会产生三类软件制品：模型、文档和代码。请说明（1）这三类软件制品之间存在什么样的关系（2 分）；（2）在开发过程中要保持这三类制品之间的一致性是指什么含义（2 分）；（3）如何来保持它们之间的一致性（2 分）。

（1）模型、文档和代码之间的关系：

- **模型**是对软件系统的一种抽象表示，用于描述系统的结构、行为和功能。模型通常包括 UML 图、数据流图等。
- **文档**是用于记录软件开发过程中的各种信息，如需求分析、设计说明、测试报告等。文档有助于团队成员之间的沟通和协作。
- **代码**是实现软件功能的计算机程序，包括源代码、库文件、配置文件等。代码是软件系统的可执行部分。

这三类软件制品之间存在密切的关系。模型为文档和代码提供了基础框架，文档记录了模型和代码的相关信息，代码则是模型和文档的实际实现。在软件开发过程中，这三者相互依赖、相互影响。

（2）保持三类制品之间的一致性是指确保模型、文档和代码在内容、结构和功能上保持一致，避免出现矛盾和冲突。这意味着当一个制品发生变化时，其他相关制品也需要相应地进行更新，以确保整个软件系统的完整性和正确性。

(3) 保持它们之间的一致性的方法:

1. ****版本控制****: 使用版本控制系统 (如 Git) 来管理模型、文档和代码的变更, 确保每个制品的历史记录清晰可追溯。
2. ****自动化工具****: 利用自动化工具 (如文档生成工具、代码生成工具) 从模型生成文档和代码, 或者根据代码更新模型和文档, 减少人为错误。
3. ****团队协作与沟通****: 加强团队内部的协作与沟通, 确保每个成员都了解制品之间的关联关系, 及时更新相关制品。
4. ****审查与测试****: 定期进行代码审查、文档审查和模型审查, 确保制品之间的一致性。同时, 通过测试来验证代码是否满足模型和文档中的要求。
5. ****持续集成与持续部署****: 实施持续集成和持续部署流程, 确保模型、文档和代码在每次迭代中都能保持一致性。

2. 简答题 (6 分)

软件测试能否发现程序中的所有错误? 为什么?

软件测试无法发现程序中的所有错误, 原因如下:

1. ****有限的测试用例****: 由于时间和资源的限制, 我们无法为程序编写和执行所有可能的测试用例。即使有无限的时间和资源, 某些特定情况下的错误可能仍然难以触发。
2. ****不可预测的用户行为****: 用户可能以意想不到的方式使用软件, 导致在正常测试过程中无法发现的错误。这些错误可能在特定的环境、配置或操作序列下才会出现。
3. ****复杂的交互和边界条件****: 软件系统中可能存在复杂的交互和边界条件, 这些情况很难完全覆盖。在某些特殊情况下, 程序可能会表现出意外的行为。
4. ****隐藏的缺陷和逻辑错误****: 程序中可能存在一些难以发现的隐藏缺陷和逻辑错误, 这些错误可能在特定条件下才会触发。由于这些错误很难预测, 因此很难通过测试来发现它们。
5. ****性能和并发问题****: 在某些情况下, 程序可能在性能和并发方面存在问题。这些问题可能与特定的硬件、操作系统或并发场景有关, 难以通过常规测试手段发现。
6. ****环境和平台差异****: 不同的环境和平台可能导致程序表现出不同的行为。在某些特定环境下, 程序可能会出现其他环境下未发现的错误。

尽管软件测试无法发现程序中的所有错误, 但通过精心设计和执行测试用例, 我们可以尽可能地发现和修复潜在的问题, 从而提高软件的质量和可靠性。同时, 结合其他质量保证手段 (如代码审查、静态分析等), 可以进一步提高软件系统的稳定性和安全性。

3. 简答题 (8 分)

结合你的课程实践, 解释说明 UML 的顺序图在分析解决和设计阶段分别用于对什么样的内容进行建模? 所建立的模型有何用途?

在我的课程实践中, UML 的顺序图在分析解决阶段和设计阶段用于对不同的内容进行建模, 具体如下:

在分析解决阶段, 顺序图用于建立系统的高级用例, 描述系统与外部实体之间的交互行为。通过顺序图, 我们可以展示系统如何响应外部事件或请求, 并确定系统所需的功能和业务逻辑。此外, 顺序图还可以用于分析系统中对象的职责和交互关系, 帮助确定系统的边界和关键业务过程。

在系统设计阶段，顺序图用于描述系统组件的详细交互和实现方式。在这个阶段，我们需要将分析阶段的结果转化为可实现的设计，顺序图可以帮助我们明确组件之间的接口和调用关系，确保系统设计的合理性和可扩展性。此外，顺序图还可以用于设计复杂的业务逻辑和算法，通过可视化的方式展示对象之间的交互过程，有助于开发人员更好地理解和管理复杂的系统设计。

通过使用顺序图进行建模，我们可以清晰地展示系统的动态行为和对象之间的交互过程，为后续的开发和测试提供指导。同时，建模的过程也有助于团队成员之间的沟通和协作，确保对系统的理解和设计的一致性

4. 分析题 (6 分)

结合“小米便签”开源代码的阅读和分析，说明该开源代码有哪些“好”的软件设计经验（3分）和“好”的程序设计经验（3分）。

小米便签的开源代码阅读和分析后，我们可以总结出以下“好”的软件设计经验和程序设计经验：

软件设计经验：

1. 用户体验优先：从用户的角度出发，考虑用户的需求和使用习惯，使软件易于使用且满足用户期望。小米便签的界面设计简洁明了，功能布局合理，使用户能够快速完成笔记的创建、编辑和查看等操作。
2. 模块化设计：将软件划分为独立的模块，每个模块负责特定的功能或业务逻辑。这有助于提高代码的可维护性和可扩展性。小米便签的代码结构清晰，各个模块职责明确，便于开发和维护。
3. 遵循开放封闭原则：软件实体（类、模块、函数等）应该是可扩展的，而不可修改的。通过将软件设计成可扩展的，可以适应未来的变化和 demand。小米便签在设计中考虑了未来的功能扩展，预留了相应的接口和扩展点。

程序设计经验：

1. 良好的编码规范：代码风格统一，命名规范，注释清晰。这有助于提高代码的可读性和可维护性。小米便签的代码规范性好，易于阅读和理解。
2. 异常处理：对可能出现的异常情况进行合理的处理，避免程序崩溃或产生不可预知的行为。小米便签在代码中充分考虑了异常情况，并进行了相应的处理。
3. 单元测试和集成测试：通过单元测试和集成测试确保代码的正确性和稳定性。小米便签的代码中包含丰富的测试用例，有助于保证软件质量。
4. 代码复用和模块化：避免重复的代码和功能，提高代码复用率。模块化设计也有助于降低代码之间的耦合度。小米便签的代码中充分体现了这一点，各个模块之间耦合度低，复用率高。
5. 安全性考虑：在代码层面考虑安全性问题，如输入验证、防止 SQL 注入等。小米便签在代码中对用户输入进行了严格的验证和处理，增强了软件的安全性。

以上是结合“小米便签”开源代码阅读和分析得出的“好”的软件设计经验和程序设计经验。这些经验可以为其他开发者提供参考和借鉴，提升自己的软件设计和程序设计能力。